

Image Processing - Lecture 5



Amir Atapour-Abarghouei

amir.atapour-abarghouei@durham.ac.uk

Reading:

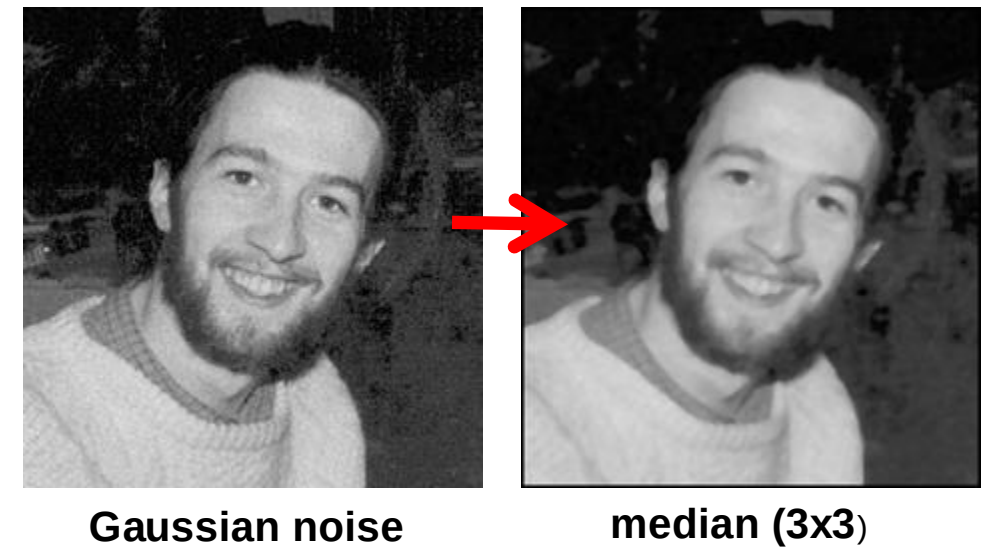
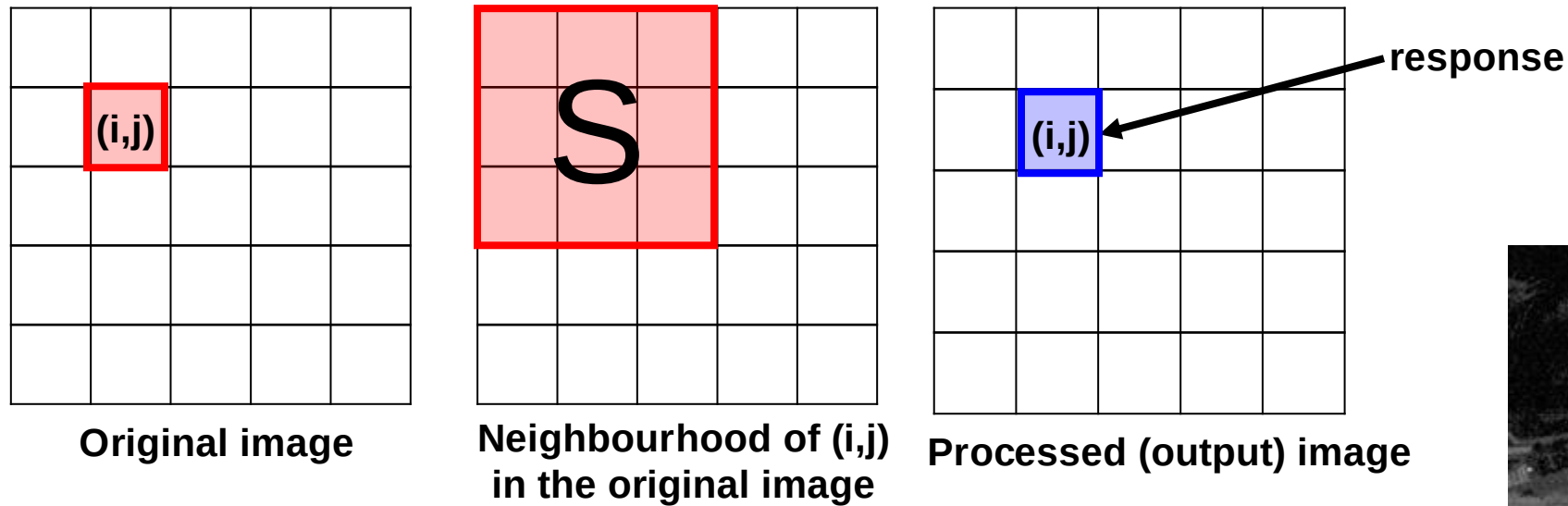
Szeliski - Sec 3.2 + 3.3.1

Gonzalez/Woods - Sec 4.2.5 + 4.8.3 + 4.9.4

Solomon/Breckon - Sec 2.2.7 + 2.2.9 + 4.4.4 + 4.5 + 4.6.1

Spatial Filtering II

Recap ... Spatial Filtering



Previously: image neighbourhoods and spatial filters

Today: Spatial **Linear** Filtering

Mask Convolution

A **mask** is a small matrix used to process another matrix (i.e. image).

- The **convolution** of an image I by a mask M , transforms each element a_{ij} of I by the following procedure:
 - Place the centre of M over $I(i, j)$
 - Do element-wise multiplication of the corresponding elements
 - Add the products

The spatial linear filtering of matrix A by mask M is called **convolution** of A by M .

- Why is this operation **linear**? It operates on the image linearly; i.e. element-wise multiplication and summation.

Convolution

Odd dimensions



For a mask of size $(2N + 1) \times (2N + 1)$, convolution can be written as:

$$I_{output}(i, j) = \sum_{k=-N}^N \sum_{l=-N}^N I_{input}(i + k, j + l) m_{kl}$$

If the mask has **odd** dimensions, it has a **well-defined centre**. Otherwise, we have to arbitrarily designate an element of the mask as its centre.

In the implementation of convolution, the matrix should be updated only after all operation results have been computed, i.e. *the matrix is not updated on the fly!*

- The mask is also called **filter**, **kernel**, **template**, **window**.

Example

- Find the convolution of A by M .

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$M = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

Example

Original image

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Processed image

..	..	..	..	..	..	..
..	1/9	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..

$$M = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

Example

Original image

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Processed image

..	..	..	..	..	..	..
..	1/9	2/9	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..

$$M = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

Example

Original image

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Processed image

..	..	..	..	..	..	..
..	1/9	2/9	2/9	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..

$$M = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

Example

Original image

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Processed image

..	..	..	..	..	..	..
..	1/9	2/9	2/9	1/9	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..

$$M = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

Example

Original image

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Processed image

..	..	..	..	..	..	..
..	1/9	2/9	2/9	1/9	0	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..

$$M = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

Example

Original image

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Processed image

..	..	..	..	..	..	..
..	1/9	2/9	2/9	1/9	0	..
..	2/9	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..

$$M = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

Example

Original image

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Processed image

..	..	..	..	..	..	..
..	1/9	2/9	2/9	1/9	0	..
..	2/9	4/9	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..

$$M = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

Example

Original image

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Processed image

..	..	..	..	..	..	..
..	1/9	2/9	2/9	1/9	0	..
..	2/9	4/9	4/9	2/9	0	..
..	2/9	4/9	4/9	2/9	0	..
..	1/9	2/9	2/9	1/9	0	..
..	..	..	..	..	..	..
..	..	..	..	..	..	..

$$M = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

Example

In this example, each pixel of the original image is replaced by the average of its neighbours.

The convolution kernel

$$M = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

gives the **mean** filter introduced in Lecture 4.

The **mean** filter is **linear**...

■ Are the median filter and conservative smoothing linear? **No...!**

Discrete Convolution in 2D

■ Discrete convolution as a 2D filter can thus be represented as follows for a generalised $N \times M$ convolution mask:

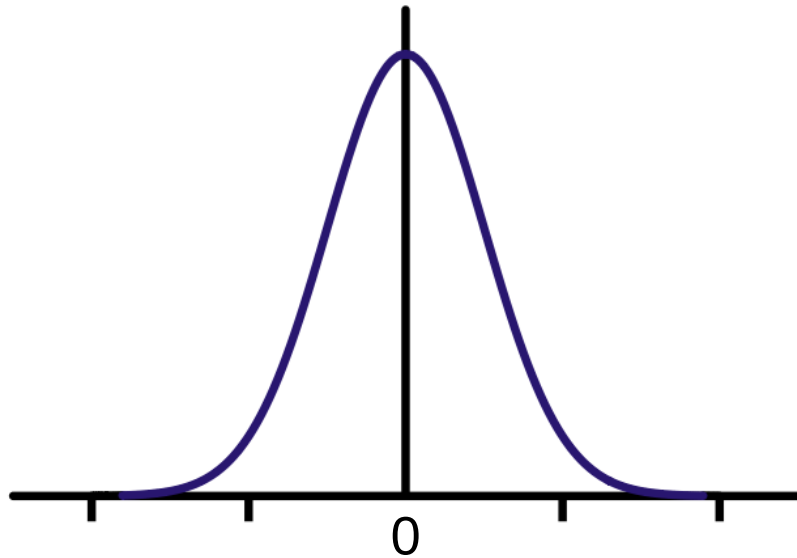
$$I_{output}(i, j) = \sum_{k=1}^N \sum_{l=1}^M I_{input}(i - (N - 1) + k, j - (M - 1) + l) m_{kl}$$

■ In terms of image processing, this is essentially the **localised weighted sum** of the image, I , and the convolution kernel, m over an $N \times M$ pixel neighbourhood, at a given location within the image (i, j) .

Convolution with Gaussian Kernels

Gaussian Filters

The value of the element p' of the mask is given by the Gaussian function, g , of the distance, $d = |p - p'|$, between the central pixel p and pixel p' .



The graph of a Gaussian function, g , has the characteristic bell shape.

Its general form is given by:

$$g(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{x^2}{2\sigma^2}}$$

where σ is the **standard deviation**, controlling the width of the bell curve.

Gaussian Filters

- used to remove image noise, but can often removes the fine features as well.
- performs blurring/smoothing of the image (Cf. **first** practical).
 - It gives more weight to the values near the centre of the mask and, gradually, less weight to the values away from the centre.



Gaussian filter



Gaussian Filters

We can construct the 3x3 Gaussian filter for $\sigma = 0.5$:

$$\begin{bmatrix} g_{\sigma}(\sqrt{2}) & g_{\sigma}(1) & g_{\sigma}(\sqrt{2}) \\ g_{\sigma}(1) & g_{\sigma}(0) & g_{\sigma}(1) \\ g_{\sigma}(\sqrt{2}) & g_{\sigma}(1) & g_{\sigma}(\sqrt{2}) \end{bmatrix}$$

- by dividing the matrix by the sum of its elements, it is normalised:

$$M_3 = \begin{bmatrix} 0.011 & 0.083 & 0.011 \\ 0.083 & 0.619 & 0.083 \\ 0.011 & 0.083 & 0.011 \end{bmatrix}$$

If the elements of a kernel **sum up to 1**, the convolution does not change the **average intensity** of the image.

Gaussian Filters

We can construct the 5x5 Gaussian filter for $\sigma = 0.5$:

$$\begin{bmatrix} g_{\sigma}(2\sqrt{2}) & g_{\sigma}(\sqrt{5}) & g_{\sigma}(2) & g_{\sigma}(\sqrt{5}) & g_{\sigma}(2\sqrt{2}) \\ g_{\sigma}(\sqrt{5}) & g_{\sigma}(\sqrt{2}) & g_{\sigma}(1) & g_{\sigma}(\sqrt{2}) & g_{\sigma}(\sqrt{5}) \\ g_{\sigma}(2) & g_{\sigma}(1) & g_{\sigma}(0) & g_{\sigma}(1) & g_{\sigma}(2) \\ g_{\sigma}(\sqrt{5}) & g_{\sigma}(\sqrt{2}) & g_{\sigma}(1) & g_{\sigma}(\sqrt{2}) & g_{\sigma}(\sqrt{5}) \\ g_{\sigma}(2\sqrt{2}) & g_{\sigma}(\sqrt{5}) & g_{\sigma}(2) & g_{\sigma}(\sqrt{5}) & g_{\sigma}(2\sqrt{2}) \end{bmatrix}$$

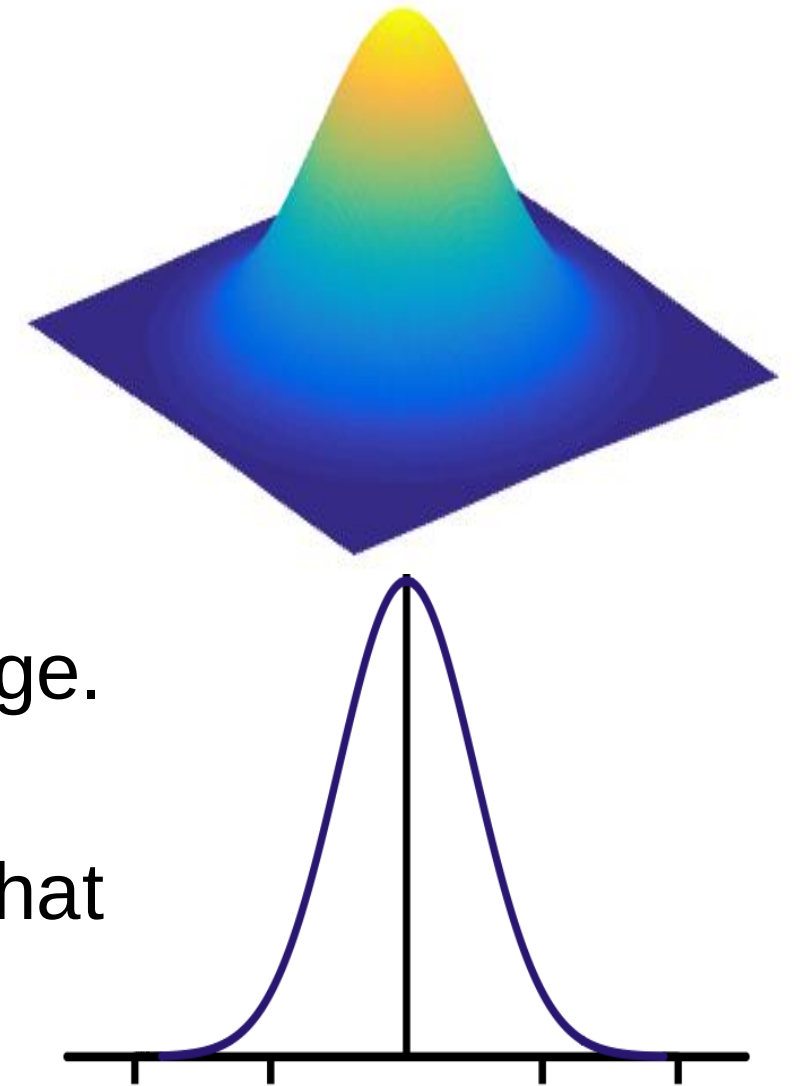
■ Normalised to:

$$M_5 = \begin{bmatrix} 0.0000 & 0.0000 & 0.0002 & 0.0000 & 0.0000 \\ 0.0000 & 0.0113 & 0.0837 & 0.0113 & 0.0000 \\ 0.0002 & 0.0837 & 0.6187 & 0.0837 & 0.0002 \\ 0.0000 & 0.0113 & 0.0837 & 0.0113 & 0.0000 \\ 0.0000 & 0.0000 & 0.0002 & 0.0000 & 0.0000 \end{bmatrix}$$

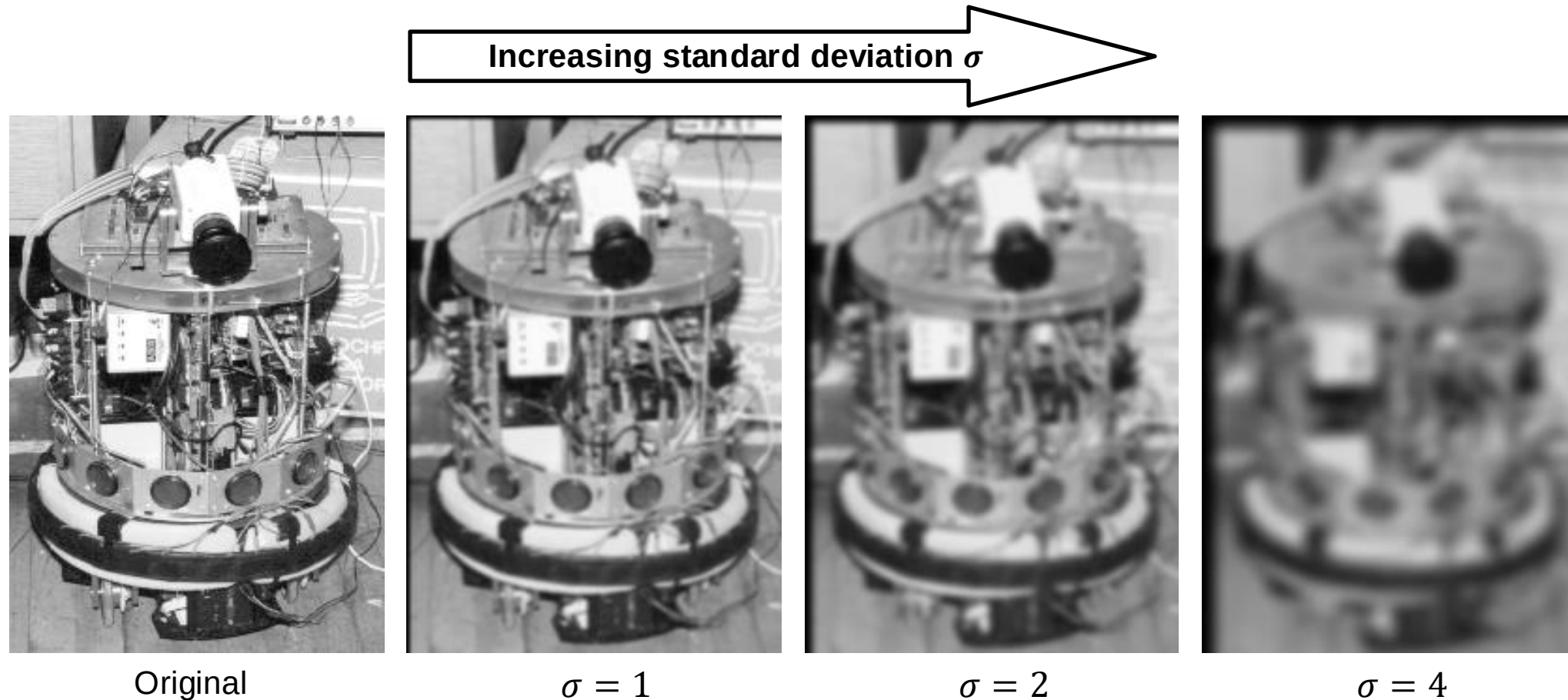
The Gaussian filter gives more weight to the values near the centre of the mask and, gradually, less weight to the values away from the centre.

Gaussian Filters

- The Gaussian function is positive everywhere.
- σ is the width of the filter
- In principle, a Gaussian mask can be infinitely large.
- We can trim small values to avoid computations that will not significantly affect the final results.



Gaussian Filtering (smoothing/blurring)



- **Example** : original (left most) then with $\sigma = 1, 2, 4$ (left to right)
 - used for noise removal + blurring effects

Gaussian Filters

- Elements of Gaussian masks come from the values of a Gaussian function.
- Allows a description of Gaussian filtering without explicit reference to masks.

$$I_p^{output} = \frac{\sum_{p' \in \Omega} g(|p - p'|) I_{p'}}{\sum_{p' \in \Omega} g(|p - p'|)}$$

- The new intensity I_p^{output} of the pixel p is a weighted sum of the intensities of the pixels p' in a neighbourhood Ω .
- The denominator of the fraction normalises the expression, making the sum of the weights 1.

Convolution using a **Laplacian** mask

**Same filtering process but ...
different mask → different effect**

Laplacian Filtering

In continuous mathematics, the **Laplacian** of a function $f(x, y)$ is defined as a sum of partial derivatives:

$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

In image processing, with the image as a function $I(x, y)$ from pixel coordinates to intensities, the Laplacian is implemented via discrete **convolutional filter**:

- Notice that the mask has some negative elements and the elements sum up to 0.

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Laplacian Filtering

How were the coefficients of the Laplacian filter computed?

- To discretise a continuous function, **derivatives become differences** and **second-order derivatives become second-order differences** (differences of differences).

- Consider a 1-dimensional image with three pixels with intensities a, b, c

$$[a \ b \ c]$$

- Consequently, the differences are:

$$[b - a, c - b]$$

Laplacian Filtering

- The difference of the intensity differences is

$$(c - b) - (b - a) = c - 2b + a$$

which is the response at the central pixel b to the convolution of the image

$$[a \ b \ c]$$

by the mask

$$[1 \ -2 \ 1]$$

Laplacian Filtering

- In 2D images, the differences of the intensity differences in the x and y directions are given by the responses to the masks

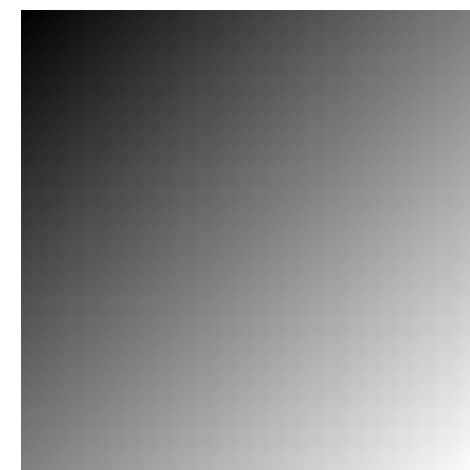
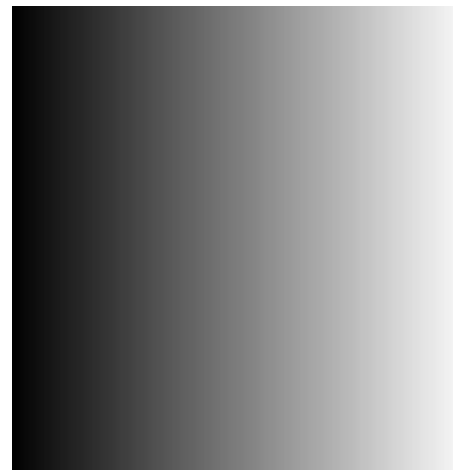
$$\begin{array}{ccc} 0 & 0 & 0 \\ 1 & -2 & 1 \\ 0 & 0 & 0 \end{array} \quad \text{and} \quad \begin{array}{ccc} 0 & 1 & 0 \\ 0 & -2 & 0 \\ 0 & 1 & 0 \end{array}$$

- By adding them we get the Laplacian filter:

$$\begin{array}{ccc} 0 & 0 & 0 \\ 1 & -2 & 1 \\ 0 & 0 & 0 \end{array} + \begin{array}{ccc} 0 & 1 & 0 \\ 0 & -2 & 0 \\ 0 & 1 & 0 \end{array} = \begin{array}{ccc} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{array}$$

Laplacian Filtering

- The response to the Laplacian filter is zero at the areas of the image where the intensity changes smoothly.
- Indeed, when **intensities** change *smoothly*, **intensity differences are equal**, and the **second order differences are zero**.
- In particular, the responses are zero at the areas of constant intensity.



The responses to the Laplacian filter of all these three images is zero everywhere.

Laplacian Filtering

- When the variation of the intensities is not smooth, the response to the Laplacian filter can be non-zero.
- The less smooth the variation of the intensities, the higher in absolute value the response of the Laplacian filter.
 - Thus, the highest in absolute value responses are at the **edges** of objects in the image.

Laplacian Filter Operation:

highlights areas of rapid intensity change i.e., edges of objects in the image.

Laplacian Filtering



Laplacian Filter



Application: Laplacian Edge Sharpening

- The Laplacian filter can be used to sharpen an image
 - by subtracting the filtered image from the original.



–



=



Application: Laplacian Edge Sharpening



Original

-



Laplacian

(normalised so we can see it)

=



■ Laplacian filter is sensitive to noise. Both edges and noise are amplified.

Solution: smooth the image with a Gaussian to remove noise and compute the Laplacian of the smoothed image - known as Laplacian of Gaussian (LoG).

Quick Demonstration before we continue ...

- Laplacian Edge Sharpening:

<https://github.com/atapour/ip-python-opencv/blob/main/laplacian.py>



Convolution

we can use varying functions or manually crafted values to define the filter to achieve varying effects on the image.

Laplacian and Gaussian filtering are just two examples.

Convolutional Neural Networks in modern machine learning are based on the same concept, except the values in the filters are “**learned**”!

Gaussian VS Median

Image with
salt & pepper noise



source: HIPR2



Gaussian smoothing



Gaussian smoothing
higher σ



Median 3×3

Key Problem: Gaussian smoothing is good at noise removal but also removes image detail (i.e. blurs).

(same problem with mean filtering too)

Solution: better noise removal filtering approaches

Bilateral Filters

Non-Local Means

Bilateral Filtering

In Gaussian filters, the influence of one pixel over another is determined by **spatial proximity only**. The nearer they are, the larger the influence.

In bilateral filters, the influence of one pixel over the other is determined by two factors:

- spatial proximity
 - a measure of **similarity between the two pixels**.
- Strictly speaking: the bilateral filter is not a spatial filter – but has a spatial component

Bilateral Filtering

Typically, the **similarity** of two pixels is measured by the **difference of their intensities** ($I_p - I_{p'}$).

Large differences of intensities mean small similarity between two pixels and thus small influence over one another in the filtering process.

We expect bilateral filtering to preserve the edges better than spatial Gaussian filtering because it is essentially “**edge aware**”.

Bilateral Filtering

Bilateral filtering is described by the equation

$$I_p^{output} = \frac{\sum_{p' \in \Omega} g_1(|p - p'|) g_2(I_p - I_{p'}) I_{p'}}{\sum_{p' \in \Omega} g_1(|p - p'|) g_2(I_p - I_{p'})}$$

- The two Gaussian functions g_1 and g_2 determine the influence of **spatial proximity** and **similarity** on the filtering process.
- The neighbourhood Ω , over which we do the summation, determines the size of the filter.
- For any pixel in Ω , the **further away** it is from p and the **more different** its intensity from that of p , the **less influence** it has over the new value of p .

Bilateral Filtering



Original



Bilateral

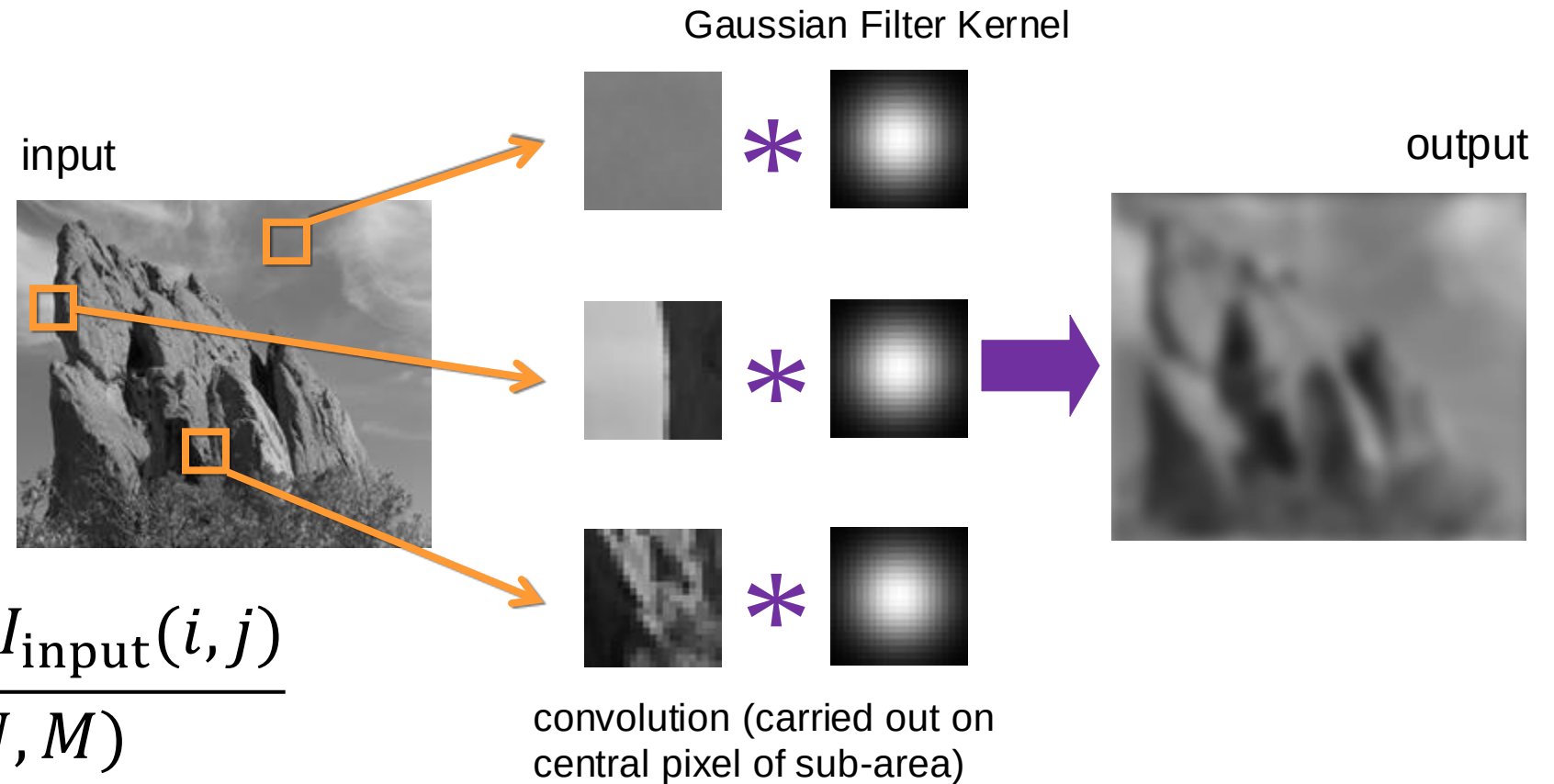


Gaussian

Gaussian Filtering

- Simple “spatial” Gaussian smoothing loses detail:

$$I_{\text{output}}(i, j) = \frac{\sum_{(N, M) \in S} W_s(N, M) I_{\text{input}}(i, j)}{\sum_{(N, M) \in S} W_s(N, M)}$$



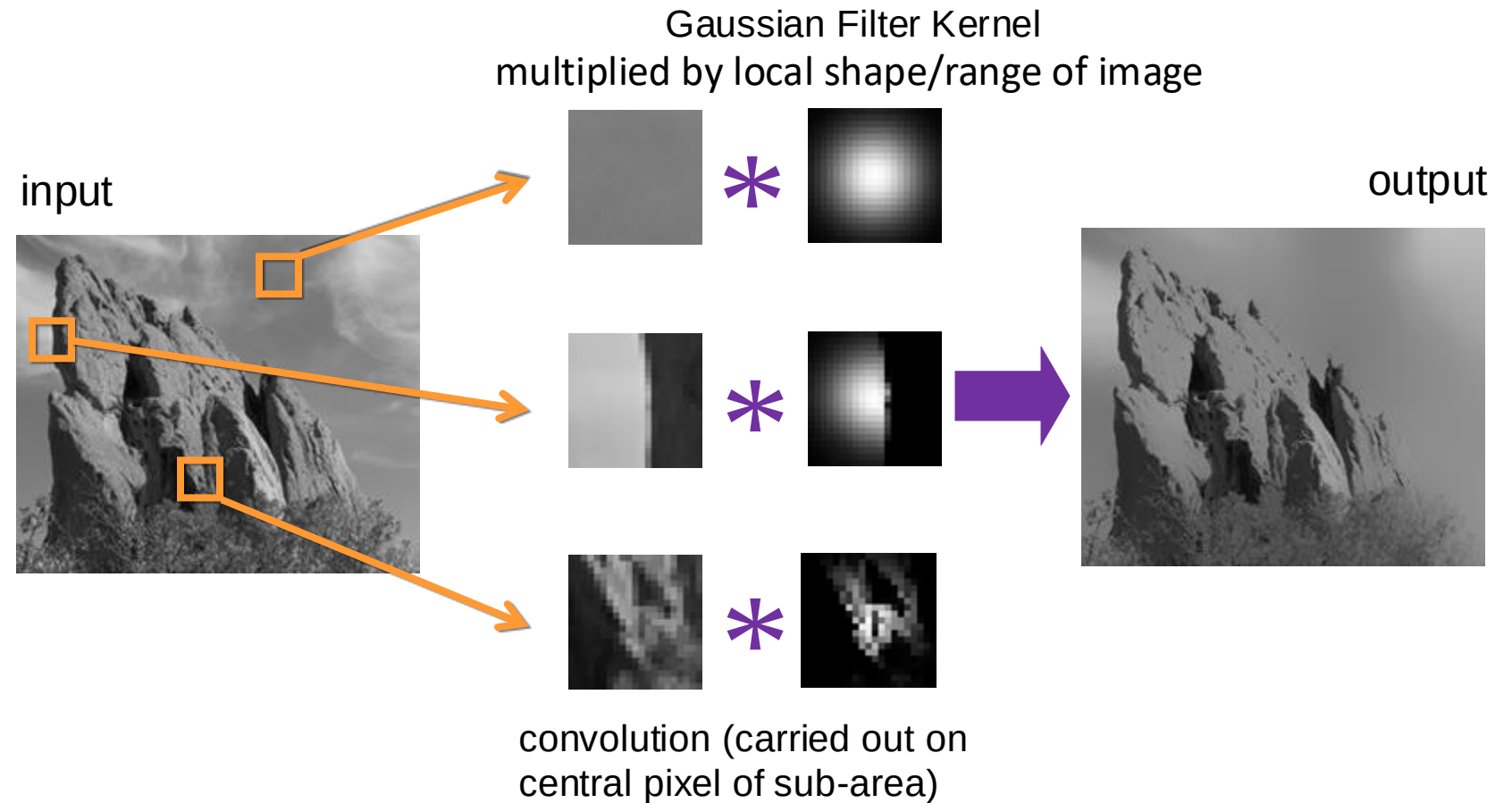
where weighting function W_s is just a 2D Gaussian kernel – this is just **Gaussian Filtering** – see earlier slides!

Bilateral Filtering

- **Improvement:**
Introduce second weighting function:

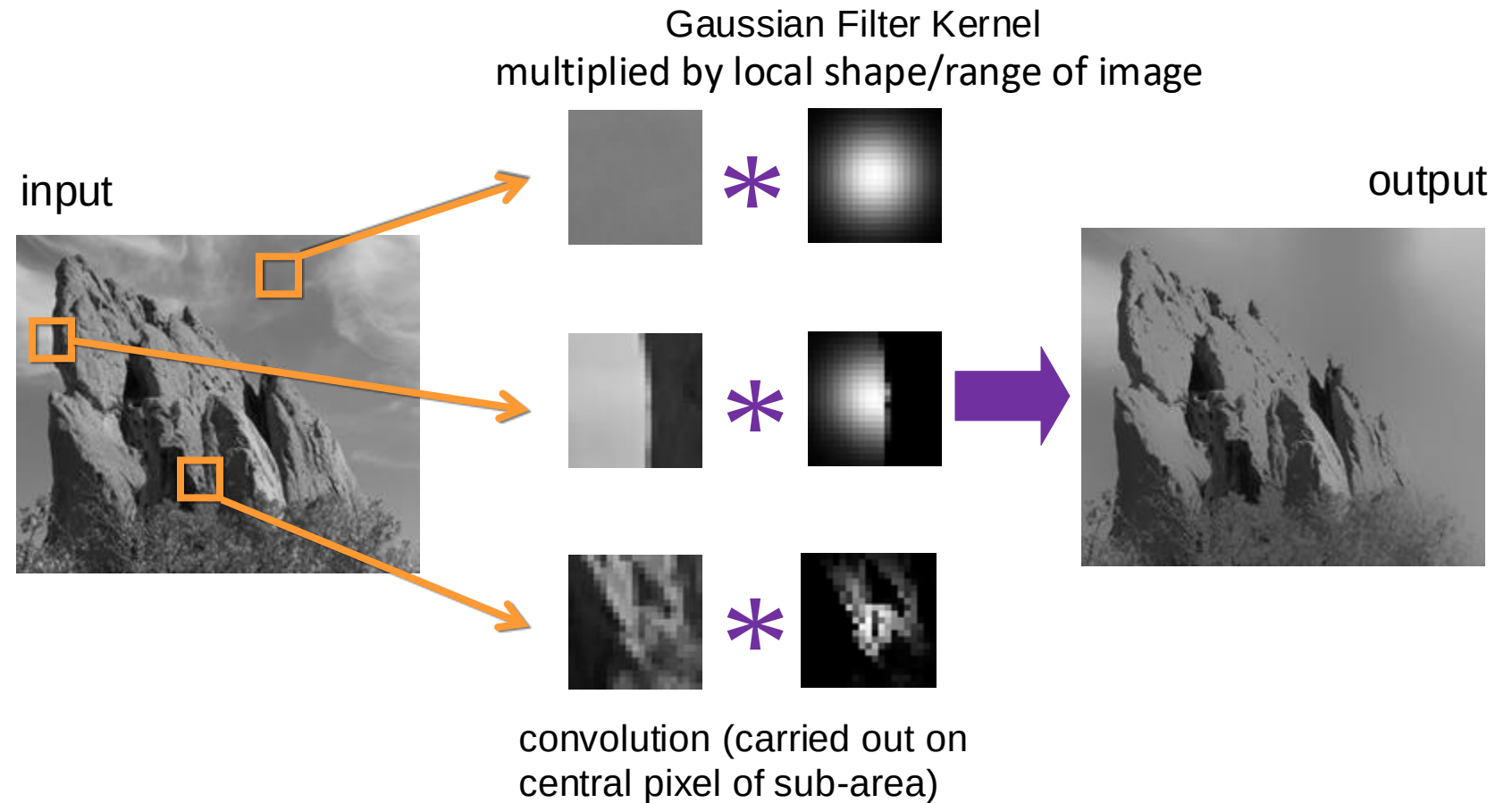
W_r

controls influence of pixels in the neighbourhood based on similarity to centre pixel (one being filtered)



Bilateral Filtering

Equivalent to multiplying the mask by a pixel-wise similarity



W_r - Desired Effect:

- **similar pixels** to the pixel being filtered → “large influence”
- **dissimilar pixels** to the pixel being filtered → “small influence”

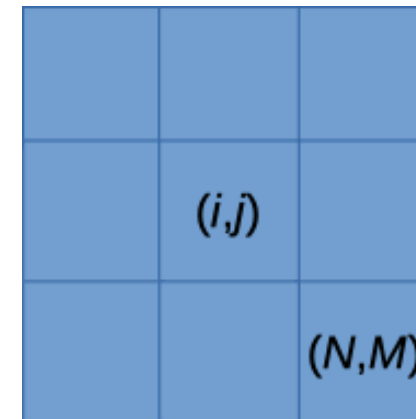
Bilateral Filtering - Details

Define function W_r as a 1D **Gaussian** defined over the **range of the two pixels**:

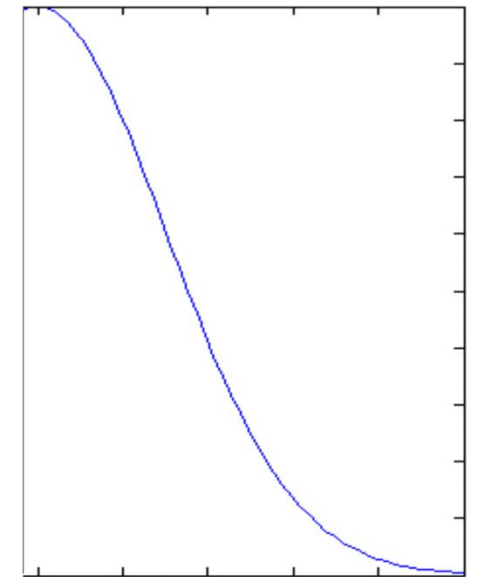
$$W_r(N, M) = G(\|I_{\text{input}}(N, M) - I_{\text{input}}(i, j)\|)$$

- Bilateral filter becomes:

$$I_{\text{output}}(i, j) = \frac{\sum_{(N, M) \in S} W_s(N, M) W_r(N, M) I_{\text{input}}(i, j)}{\sum_{(N, M) \in S} W_s(N, M) W_r(N, M)}$$



$G()$



0 $\longrightarrow$ 255

$\|I_{\text{input}}(N, M) - I_{\text{input}}(i, j)\|$

Magnitude of difference
in pixel values

- W_r controls the influence of each pixel based local range
- W_s is Gaussian filter (standard as per earlier slides)

Bilateral Filtering



Original Image

σ_1 and σ_2 are the
standard deviations
of g_1 and g_2

σ_1 parameter for W_s
 σ_2 parameter for W_r

$\sigma_1 = 2$

$\sigma_2 = 0.1$

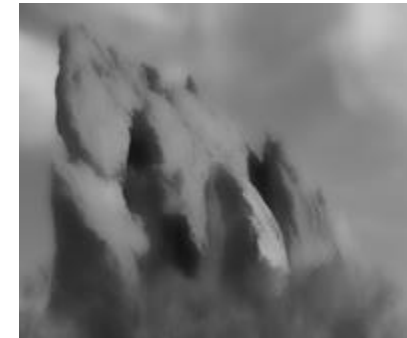
$\sigma_2 = 0.25$

(Gaussian blur)

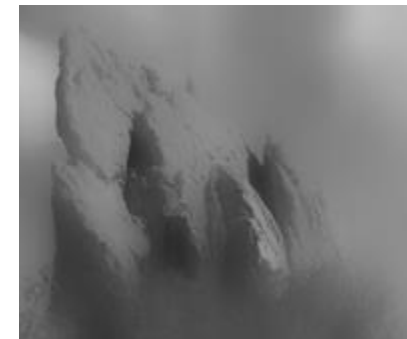
$\sigma_2 = \infty$



$\sigma_1 = 6$

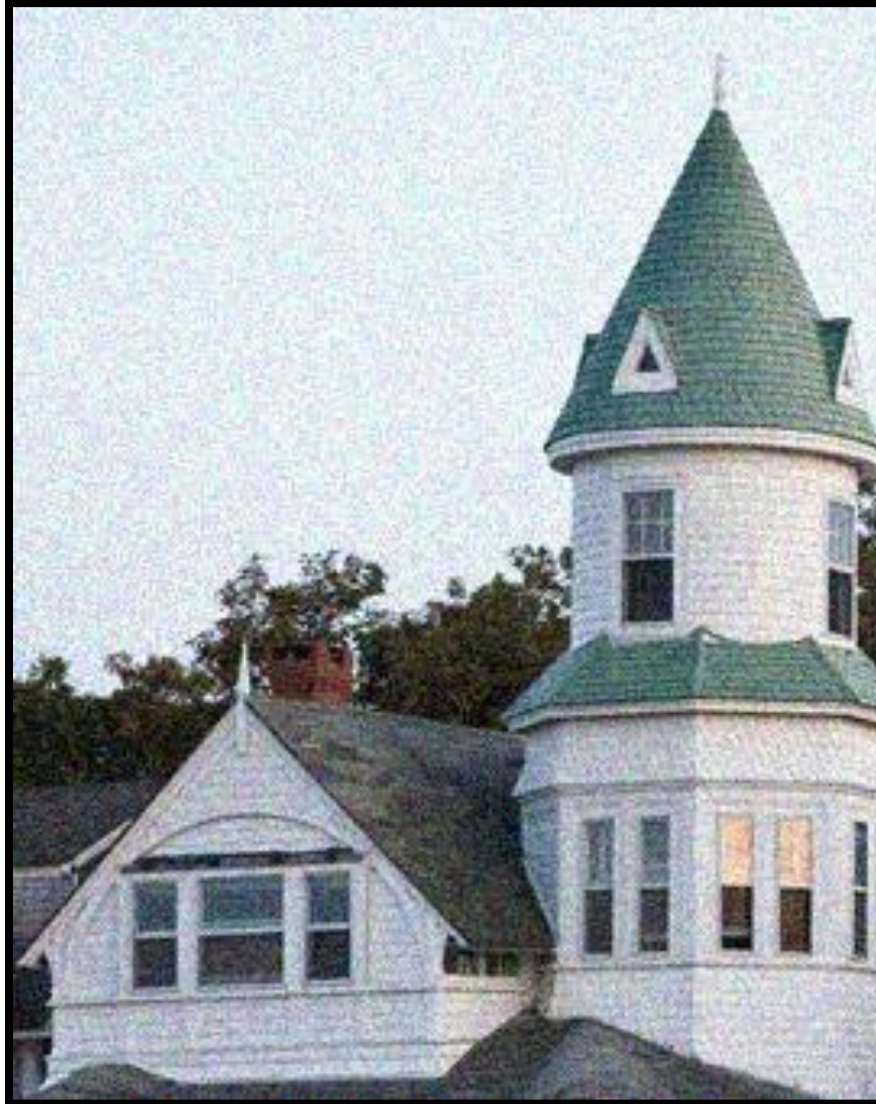


$\sigma_1 = 18$



Bilateral Filtering - Examples

Noisy input

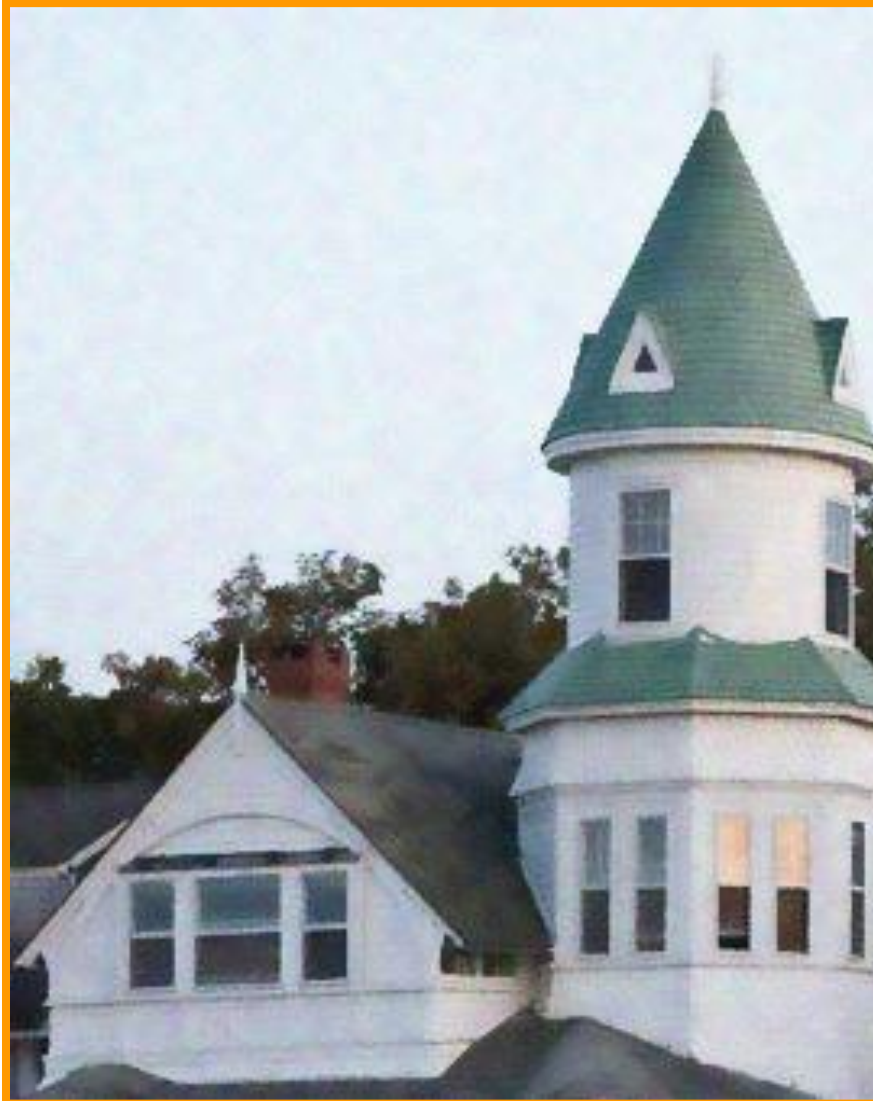


Bilateral filter



Bilateral Filtering - Examples

Bilateral filter

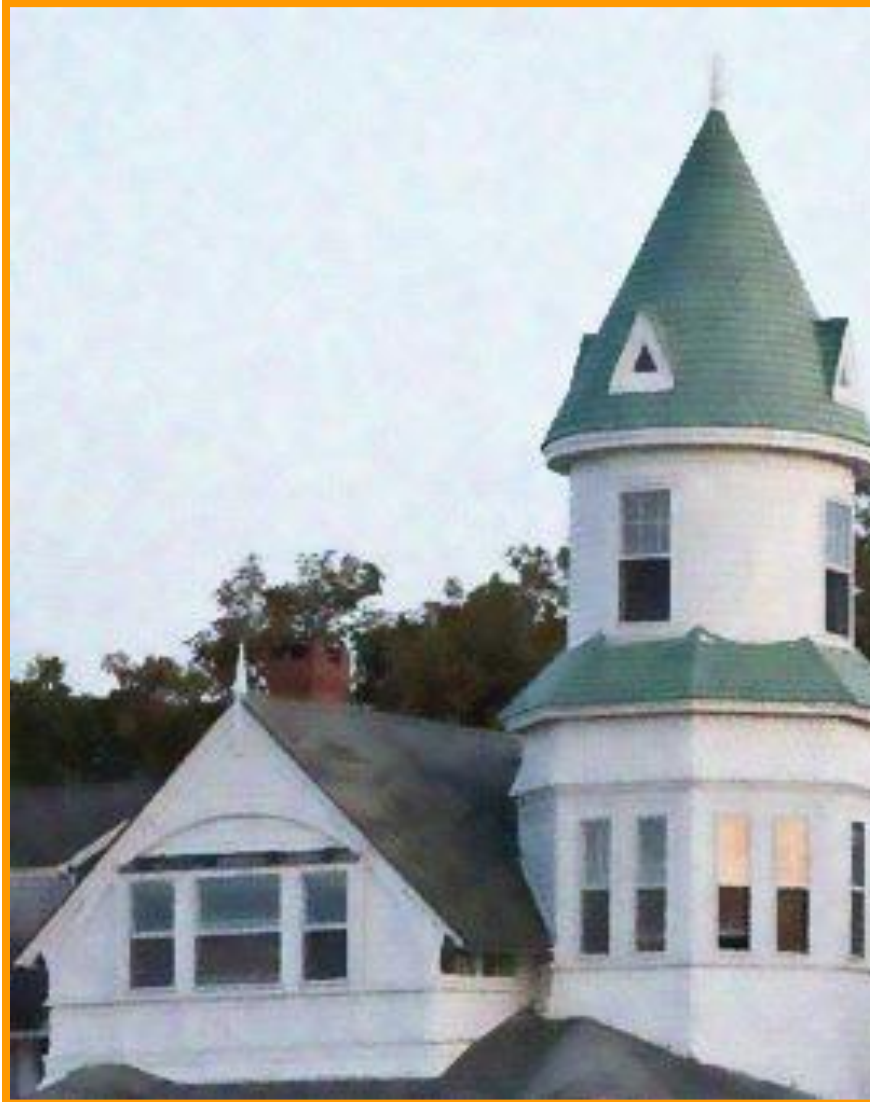


Bilateral filter higher σ_1



Bilateral vs Median

Bilateral filter



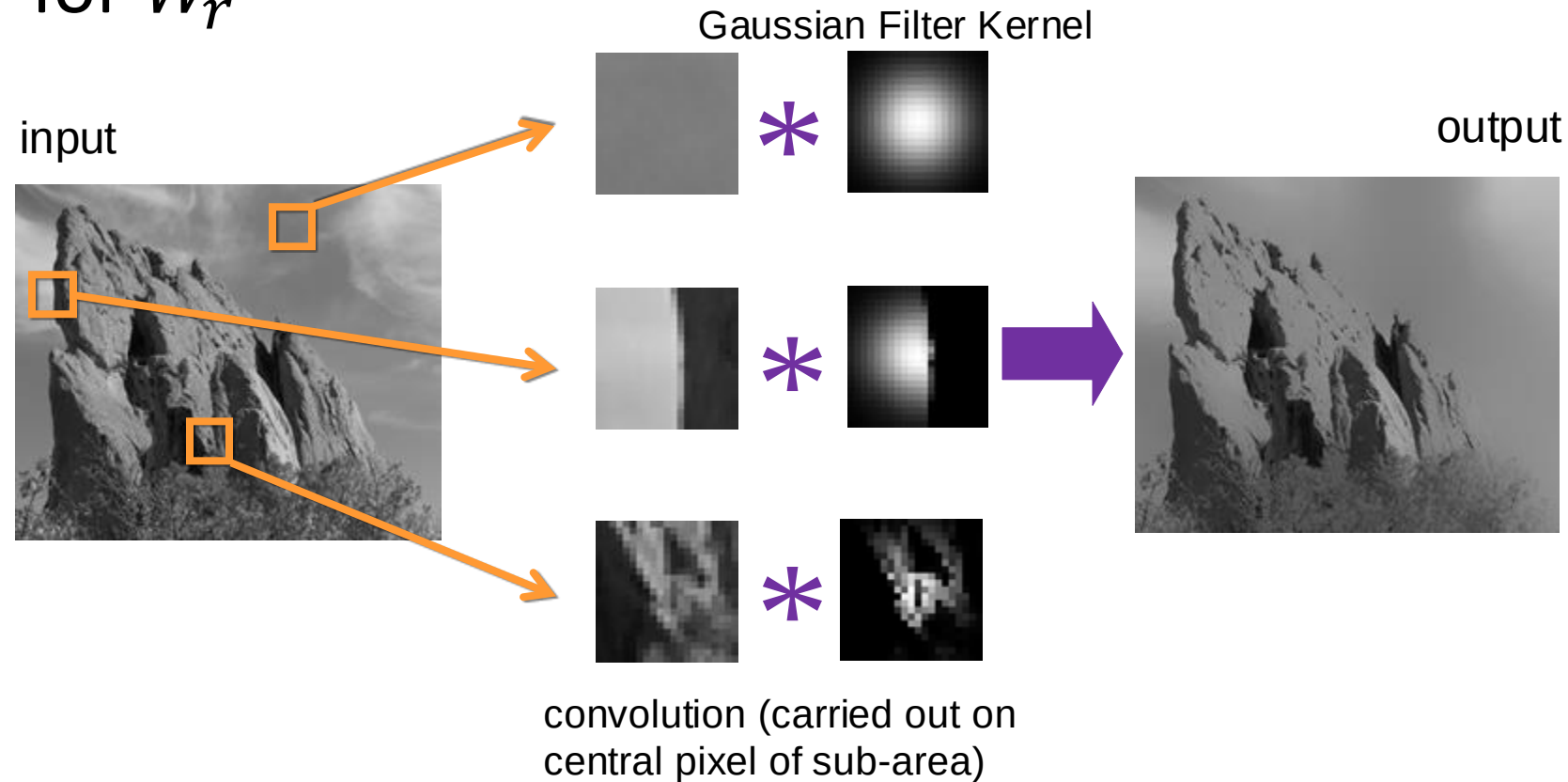
Median 3x3



Bilateral Filtering

σ_1 parameter for W_s

σ_2 parameter for W_r



Notes

- **Variants:** can also use with alternative functions for W_s and W_r (not just Gaussian!)
- Good for Gaussian noise, but **what about Salt n' Pepper?**

Quick Demonstration

- Bilateral Filtering:

https://github.com/atapour/ip-python-opencv/blob/main/bilateral_filter.py



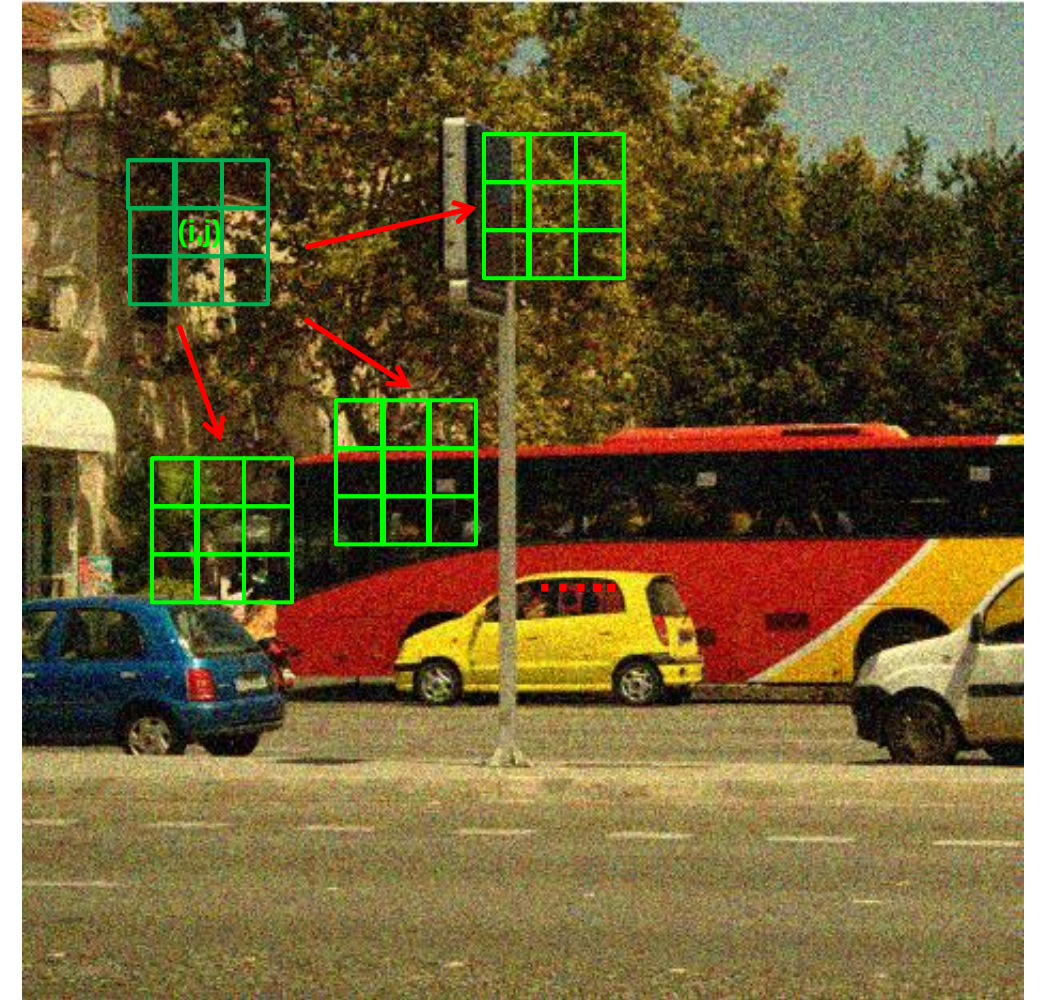
Noise Removal Filtering ...

The Return of the Mean

Non-Local Means

Key Concept:

- Search the image for **neighbourhoods** similar to the one of the pixel being processed.
- Replace the current pixel with a **weighted mean** of pixels with similar neighbourhoods.
- **Weights** computed according to **similarity between neighbourhoods**.
- The averaging spans pixels that are **not local** within the image to the target pixel.
 - *hence the name!*



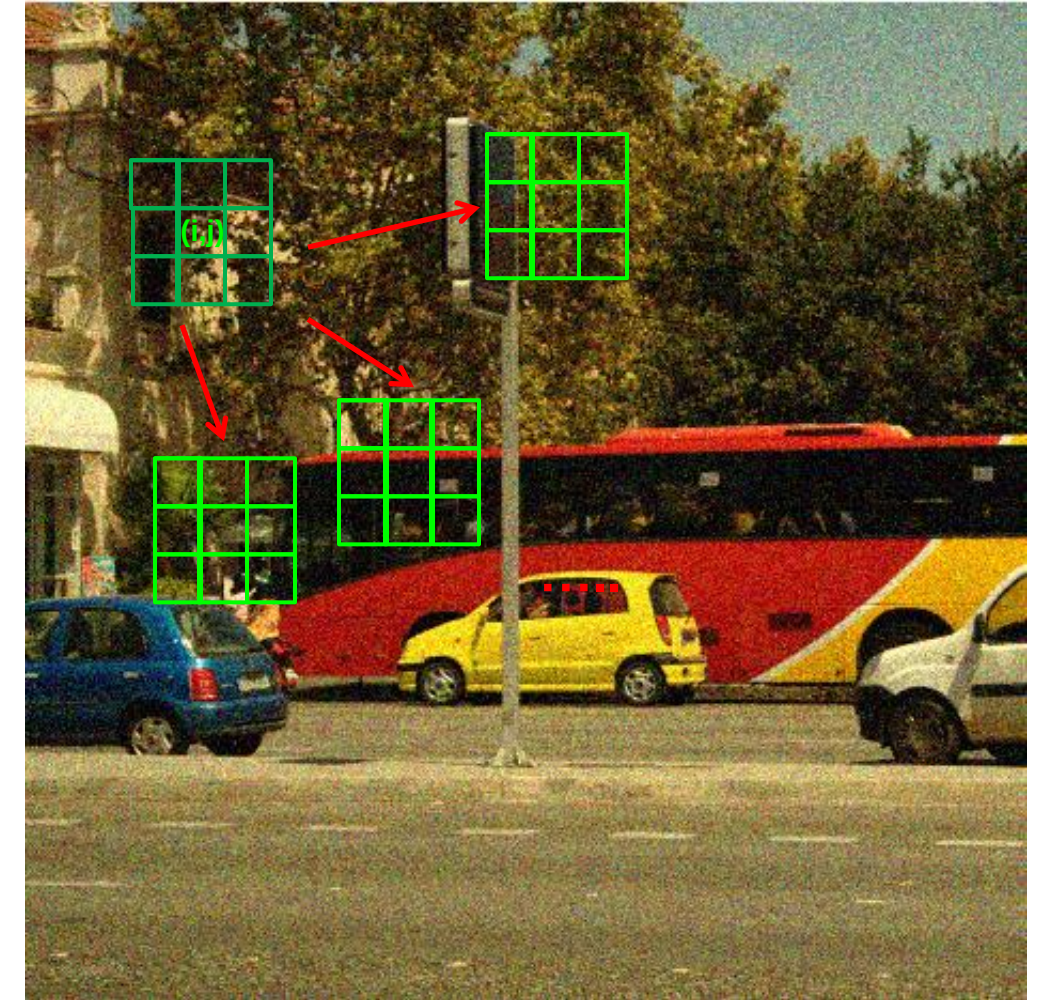
Neighbourhoods illustrated
larger than reality!

Non-Local Means

Output pixel = weighted sum of other pixels (all, or a subset of them) weighted by a similarity measure of the corresponding neighbourhoods.

$$I_{\text{output}}(i, j) = \frac{\sum W_{\text{neighbourhood}}(k, l) I_{\text{input}}(k, l)}{\sum W_{\text{neighbourhood}}(k, l)}$$

- Similarity = Normalised Euclidean Distance between neighbourhoods.



$$W_{\text{neighbourhood}}(k, l) = e^{\frac{-((\text{neighbourhood}(k, l) - \text{neighbourhood}(i, j))^2)}{h^2}}$$

Non-Local Means



Gaussian Noise



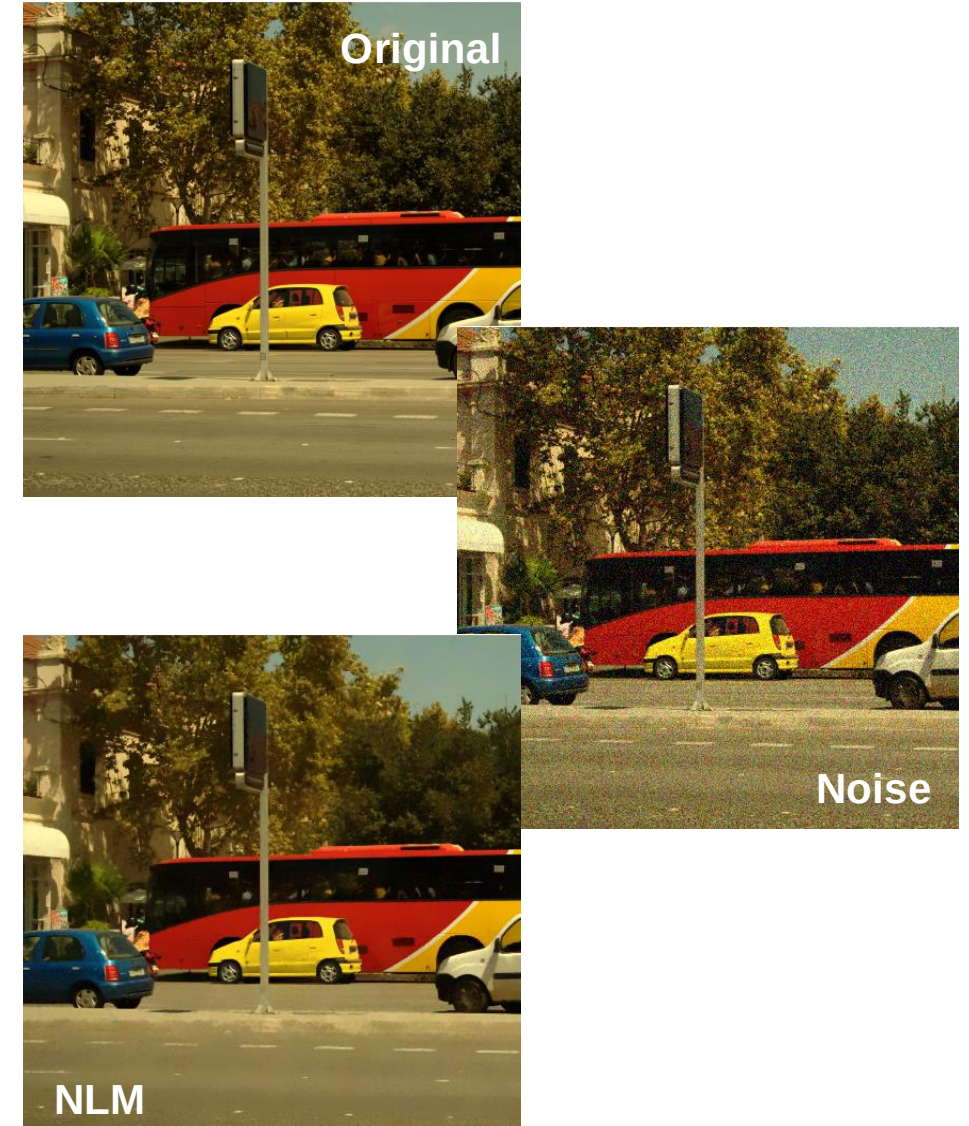
Non-Local Means

Non-Local Means

Close to state-of-the-art noise removal for photographic imagery.

Performance:

- good for noise removal.
- dependent on neighbourhood size $N \times N$
- slow, but several optimisations exist.
- easily parallelised.



Quick Demonstration

- Non-Local Means Filtering:

https://github.com/atapour/ip-python-opencv/blob/main/nlm_filter.py



2D convolution – General Side Notes

Image boundaries – what do we do here?

- use padding with zeros or another value (can lead to visually poor results)
- extrapolate/mirror values from inside the image (one of the better options)
- do not perform convolution in these regions
(the result will be smaller than the original – not desirable).

?	?	?	?	?	?	?	0.4	0.4	0.0	0.6	0.3	0.2	0.2	0.0	0.0	0.0	0.0	0.0	0.0
?	0.4	0.0	0.6	0.3	0.2	?	0.4	0.4	0.0	0.6	0.3	0.2	0.2	0.0	0.4	0.0	0.6	0.3	0.2
?	0.8	0.3	0.7	0.2	0.7	?	0.8	0.8	0.3	0.7	0.2	0.7	0.7	0.0	0.8	0.3	0.7	0.2	0.7
?	0.5	0.4	0.7	0.5	0.3	?	0.5	0.5	0.4	0.7	0.5	0.3	0.3	0.0	0.5	0.4	0.7	0.5	0.3
?	0.5	0.3	0.6	0.8	0.7	?	0.5	0.5	0.3	0.6	0.8	0.7	0.7	0.0	0.5	0.3	0.6	0.8	0.7
?	0.9	0.3	0.0	0.7	0.2	?	0.9	0.9	0.3	0.0	0.7	0.2	0.2	0.0	0.9	0.3	0.0	0.7	0.2
?	?	?	?	?	?	?	0.9	0.9	0.3	0.0	0.7	0.2	0.2	0.0	0.0	0.0	0.0	0.0	0.0

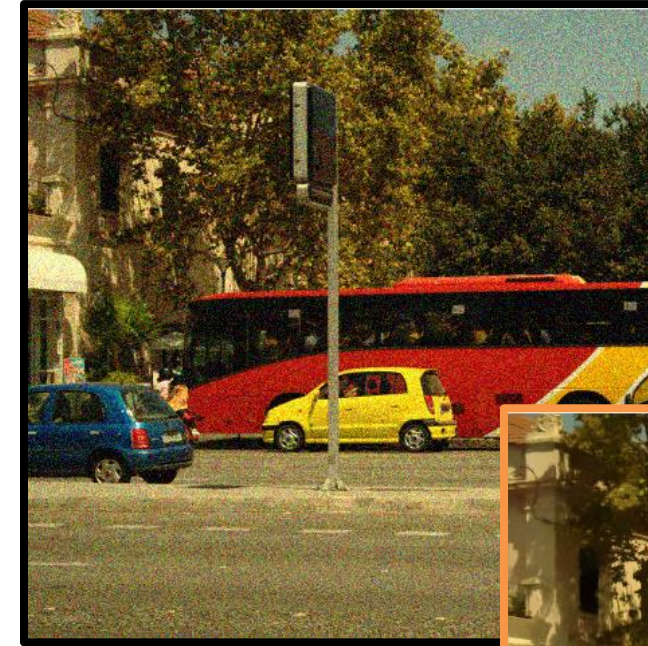
or

Computational Cost: expensive but not too bad with modern hardware

- 9 multiplication and 8 additions for a single position, for 3×3 kernel.
- N^2 multiplications and $N^2 - 1$ additions for $N \times N$ kernel.
- $C \times R$ image convolution = $O(CRN^2)$ for $N \times N$ kernel.

Summary

- Convolution in theory and in practice
- Gaussian filtering
- Laplacian filtering
- Bilateral filtering
- Non-Local Means



Assignment!

- On the module page on Ultra!
- Real-world application...
- Read the instructions carefully.
- Post any questions you have in the discussion board on Ultra
 - **Assessment → Image Processing Questions/Answers**

